

SLAM-Based Mapping Robot

Perception & Kinematics

Name: Kinyua Seaman

Module: Robotics — MSc Artificial Intelligence

Date: May 2026

Project Type: Simulation-Based Coursework

Contents

1	Introduction	2
1.1	Aims and Learning Outcomes	2
2	Theoretical Background	2
2.1	Differential-Drive Kinematics	2
2.2	Odometry Noise Model	3
2.3	LiDAR Sensor Simulation	3
2.4	Log-Odds Occupancy Grid Mapping	3
2.5	Bresenham’s Line Algorithm	4
3	System Architecture	4
3.1	Module Descriptions	4
3.1.1	<code>environment.py</code> — World & Sensor	4
3.1.2	<code>robot.py</code> — Kinematics & Odometry	5
3.1.3	<code>slam.py</code> — Occupancy Grid Mapping	5
3.1.4	<code>main.py</code> — Simulation Orchestrator	5
3.1.5	<code>visualization.html</code> — Interactive Web Demo	5
4	Implementation Details	5
4.1	Waypoint Navigation Controller	5
4.2	Sensor–Map Integration	6
4.3	Log-Odds Saturation Considerations	6
5	Experimental Results	6
5.1	Simulation Progression	6
5.2	Quantitative Evaluation	8
5.3	Qualitative Observations	9
6	Critical Analysis	9
6.1	Strengths	9
6.2	Limitations	9
6.3	Comparison with Full SLAM Solutions	10
7	Future Work & Extensions	10
7.1	Immediate Improvements	10
7.2	Advanced Extensions	10

8 Repository Structure	11
9 Conclusion	11

1 Introduction

Simultaneous Localisation and Mapping (SLAM) is a fundamental problem in autonomous robotics: a mobile agent must incrementally construct a map of an unknown environment while simultaneously tracking its own position within that map. The problem is circular—accurate mapping requires a good pose estimate, yet accurate localisation requires a good map—and has been studied extensively since the seminal work of Smith, Self and Cheeseman (1990) and the probabilistic formulation by Thrun, Burgard and Fox (2005).

This project implements a **custom, from-scratch 2D SLAM simulation** in Python that explicitly models:

1. **Differential-drive kinematics** with stochastic odometry noise,
2. **2D LiDAR ray-casting** with configurable Gaussian sensor noise, and
3. **Log-odds occupancy grid mapping** using Bresenham’s line algorithm for efficient ray-to-cell projection.

The simulation was developed without ROS/Gazebo, requiring every subsystem—physics, sensing, mapping, and visualisation—to be hand-coded, thereby deepening engagement with the underlying mathematics.

1.1 Aims and Learning Outcomes

Learning Outcome	How Addressed
Probabilistic Robotics	Log-odds occupancy grid; Gaussian motion & observation noise models
Sensor Fusion	Noisy odometry pose fused with LiDAR depth scans to infer grid state
Coordinate Frame Management	Local robot frame (LiDAR bearings) $\leftrightarrow$ global map frame (occupancy grid indices)
Motion Planning (basic)	Proportional waypoint-following controller

2 Theoretical Background

2.1 Differential-Drive Kinematics

The robot is modelled as a differential-drive platform governed by the velocity motion model (Thrun et al., 2005, Ch. 5). Given linear velocity v and angular velocity ω , the continuous-time kinematic equations in the global frame are:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega \quad (1)$$

For the discrete time step Δt , two cases are handled:

Pure translation ($\omega = 0$):

Euler integration is applied directly.

Arc motion ($\omega \neq 0$):

The exact closed-form arc equations are used:

$$x_{t+1} = x_t - \frac{v}{\omega} \sin \theta_t + \frac{v}{\omega} \sin(\theta_t + \omega \Delta t) \quad (2)$$

$$y_{t+1} = y_t + \frac{v}{\omega} \cos \theta_t - \frac{v}{\omega} \cos(\theta_t + \omega \Delta t) \quad (3)$$

$$\theta_{t+1} = \theta_t + \omega \Delta t \quad (4)$$

This closed-form integration is more accurate than naïve Euler for non-zero curvature and avoids the systematic drift that plagues first-order approximations over long trajectories.

2.2 Odometry Noise Model

Noisy odometry is generated by corrupting the commanded velocities:

$$\hat{v} = v + \varepsilon_v, \quad \varepsilon_v \sim \mathcal{N}(0, \alpha_1 v^2 + \alpha_2 \omega^2) \quad (5)$$

$$\hat{\omega} = \omega + \varepsilon_\omega, \quad \varepsilon_\omega \sim \mathcal{N}(0, \alpha_3 v^2 + \alpha_4 \omega^2) \quad (6)$$

The noise parameters $\alpha_1 \dots \alpha_4$ control the velocity-dependent covariance structure. This is the standard four-parameter model from Thrun et al. (2005, Table 5.6), ensuring that noise scales realistically with commanded motion—faster speeds and sharper turns produce proportionally larger odometry errors. A small constant (10^{-6}) is added inside the square root for numerical stability when both velocities are near zero.

2.3 LiDAR Sensor Simulation

The simulated 2D LiDAR casts $N = 72$ rays uniformly over a full 2π field of view. Each ray is traced incrementally through the occupancy grid at half-resolution step size (0.5 m) until either:

- An **occupied cell is hit**: the distance is recorded plus additive Gaussian noise $\varepsilon_d \sim \mathcal{N}(0, \sigma_d)$, or
- The **maximum range** (15 m) is reached: a max-range reading is returned.

Note: The ray-casting operates in the **global frame** using the robot's *true* pose, not the estimated pose. This is physically correct: the sensor measures the real world. The noisy estimated pose is used only when integrating these readings into the map—faithfully modelling the effect of localisation error on map quality.

2.4 Log-Odds Occupancy Grid Mapping

The occupancy grid represents the environment as a discretised 2D array where each cell $m_{i,j}$ stores the log-odds of occupancy:

$$l_{i,j} = \log \frac{P(m_{i,j} = 1 \mid z_{1:t})}{P(m_{i,j} = 0 \mid z_{1:t})} \quad (7)$$

Under a static-world assumption and conditional independence of observations given the map, the Bayesian update in log-odds form reduces to a simple additive update (avoiding costly multiplications and normalisations):

$$l_{i,j}^t = l_{i,j}^{t-1} + l_{\text{inv}} - l_0 \quad (8)$$

Symbol	Value	Meaning
l_{occ}	$\log \frac{0.7}{0.3} \approx 0.847$	Inverse-sensor log-odds for occupied cells
l_{free}	$\log \frac{0.3}{0.7} \approx -0.847$	Inverse-sensor log-odds for free cells
l_0	0.0	Prior log-odds (maximum uncertainty)

The grid is initialised to $l_0 = 0$ (50% prior probability). Repeated observations drive cells towards strong occupied or free beliefs. The final probability is recovered via the logistic function:

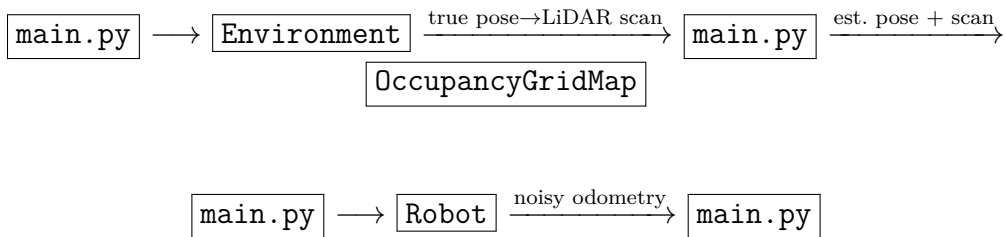
$$P(m_{i,j} = 1) = 1 - \frac{1}{1 + \exp(l_{i,j})} \quad (9)$$

2.5 Bresenham’s Line Algorithm

To determine which grid cells each LiDAR ray passes through, the implementation uses **Bresenham’s line algorithm**—an integer-arithmetic rasterisation method that enumerates all cells along a line from the sensor origin to the ray endpoint. All intermediate cells are marked as free ($+l_{\text{free}}$) and the terminal cell (if a hit occurred) is marked as occupied ($+l_{\text{occ}}$). This is computationally efficient (no floating-point divisions per step) and avoids the aliasing artifacts that simpler stepping methods can introduce.

3 System Architecture

The project follows a modular, object-oriented design with clear separation of concerns. The data flow is:



3.1 Module Descriptions

3.1.1 environment.py — World & Sensor

Responsibility	Details
Grid construction	50×50 m world at 1 m resolution → 50×50 cell array
Obstacle generation	Outer boundary walls + 3 fixed inner walls + 5 random rectangular blocks
LiDAR simulation	<code>get_lidar_scan</code> : ray-casting with configurable max range, ray count, FOV, and noise

3.1.2 `robot.py` — Kinematics & Odometry

Responsibility	Details
True motion	Exact arc-based differential-drive update
Noisy odometry	Velocity-dependent Gaussian corruption (α_1 – α_4)
Path tracking	Maintains separate <code>true_path</code> and <code>estimated_path</code> histories
Visualisation helper	<code>plot_robot</code> : renders a circle + heading line

3.1.3 `slam.py` — Occupancy Grid Mapping

Responsibility	Details
Log-odds grid	Initialised to 0 (uniform prior)
Bayesian update	<code>update</code> : iterates over LiDAR rays, applies Bresenham, updates cells
Bresenham rasterisation	<code>bresenham</code> : generator yielding (x, y) integer grid coordinates
Probability conversion	<code>get_map_probabilities</code> : logistic transform for visualisation

3.1.4 `main.py` — Simulation Orchestrator

Responsibility	Details
Waypoint navigation	8 predefined waypoints with proportional heading controller
Simulation loop	Up to 1500 steps at $\Delta t = 0.1$ s
Visualisation	Dual-panel matplotlib figure saved every 50 steps
Evaluation	Terminal MSE between true and estimated (x, y) positions

3.1.5 `visualization.html` — Interactive Web Demo

A standalone HTML/Canvas/JavaScript re-implementation of the full pipeline (environment, robot, occupancy grid, Bresenham) with real-time rendering and user-adjustable sliders for linear speed, angular speed, and sensor noise.

4 Implementation Details

4.1 Waypoint Navigation Controller

The robot is driven by a simple **proportional controller**:

```
target_theta = np.arctan2(dy, dx)
angle_diff = robot.normalize_angle(target_theta - rtheta)

v = min(2.0, distance) if abs(angle_diff) < np.pi/4 else 0.0
w = max(-1.0, min(1.0, angle_diff * 2.0))
```

Key design choices:

- **Heading-gated linear velocity:** The robot only drives forward when the heading error is within $\pm 45^\circ$, preventing large lateral drift during turns.
- **Clamped angular velocity:** $\omega \in [-1, 1]$ rad/s prevents physically unrealistic instantaneous rotation.
- **Speed proportional to distance:** $v = \min(2.0, d)$ provides natural deceleration as waypoints are approached, reducing overshoot.

4.2 Sensor–Map Integration

A critical architectural decision is the separation of sensing and mapping:

```
# LiDAR scan from TRUE pose (physically correct)
rel_angles, distances = env.get_lidar_scan(robot.true_pose,
    num_rays=72)

# Map update using ESTIMATED pose (simulates real-world SLAM)
slam.update(robot.estimated_pose, rel_angles, distances)
```

This faithfully models the SLAM problem: the sensor reads the real world, but the robot can only use its (noisy) odometry estimate when projecting those readings into the map. The resulting map quality directly reflects localisation accuracy.

4.3 Log-Odds Saturation Considerations

The current implementation does not clamp log-odds values. Over many observations, cells can accumulate arbitrarily large positive or negative log-odds, which:

- **Advantage:** Strongly observed cells are virtually permanent, providing robust map stability.
- **Disadvantage:** In a dynamic environment, it would take many contradictory observations to reverse a strongly held belief. For static-world assumptions (as in this project), this is acceptable.

5 Experimental Results

5.1 Simulation Progression

The simulation was executed with seed 42 for reproducibility. The following figures show the dual-panel output at three stages of exploration.

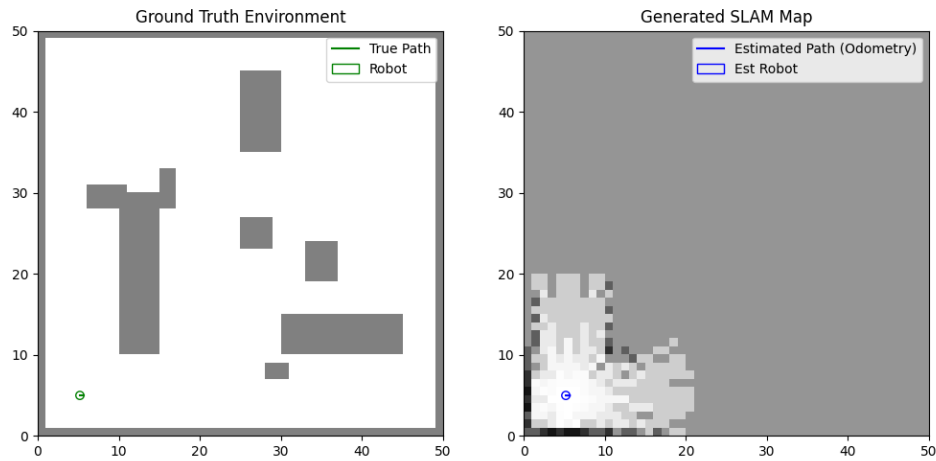


Figure 1: Initial state (Step 0). The robot is at position (5, 5) with an empty occupancy grid.

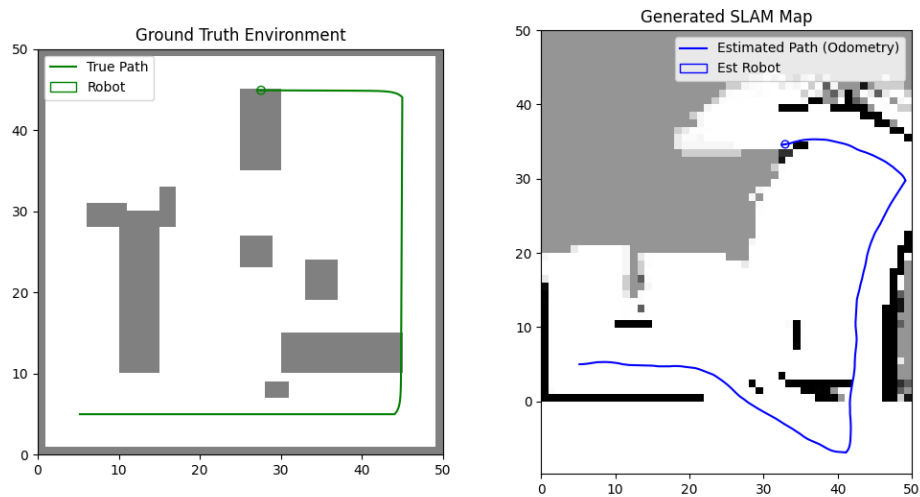


Figure 2: Mid-exploration (Step 500). The occupancy grid shows partially reconstructed walls with visible distortion from accumulated odometry drift.

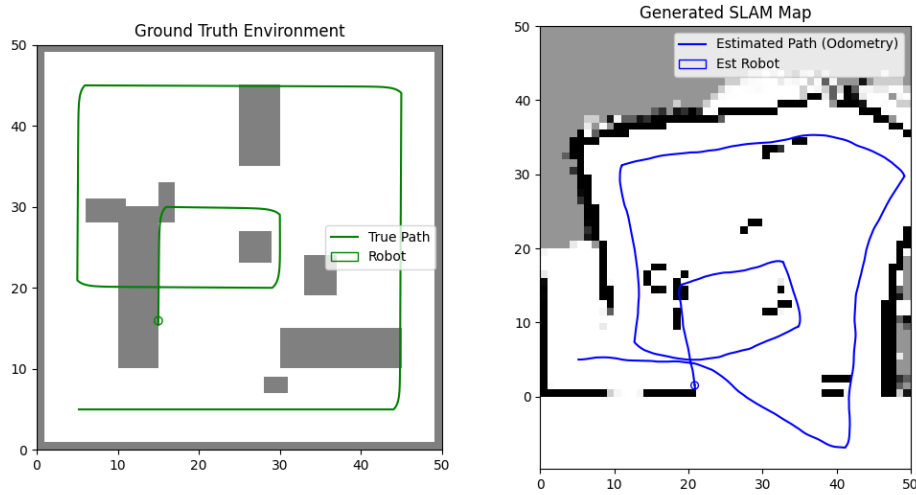


Figure 3: Final state (Step 1095). Left: true environment and true path (green). Right: SLAM-generated occupancy grid with estimated path (blue). Note the progressive divergence between the two paths due to odometry drift.

5.2 Quantitative Evaluation

The localisation error is measured as the **Mean Squared Error** between the true and estimated positions over all time steps:

$$\text{MSE} = \frac{1}{T} \sum_{t=1}^T \left[(x_t - \hat{x}_t)^2 + (y_t - \hat{y}_t)^2 \right] \quad (10)$$

The simulation reports this at completion, providing a single scalar metric of odometry drift severity. By design—without any pose-correction mechanism (e.g., scan matching, loop closure)—this error is expected to grow monotonically, demonstrating the fundamental insufficiency of dead-reckoning alone.

5.3 Qualitative Observations

Observation	Analysis
Wall structure is recognisable	Despite odometry drift, the log-odds formulation accumulates sufficient evidence to reveal major structures (boundaries, large obstacles).
Map distortion increases with distance	The estimated path diverges progressively from the true path, causing later map updates to project rays from incorrect positions. This manifests as “smeared” or “doubled” walls in the occupancy grid.
Small obstacles are less reliably mapped	Random blocks observed from only a few poses accumulate fewer log-odds updates and may appear faint or misaligned.
Free-space mapping is robust	The Bresenham ray-tracing ensures that corridors the robot traverses are reliably marked as free, even with moderate pose error.

6 Critical Analysis

6.1 Strengths

1. **Full-stack implementation:** Every component—kinematics, sensing, mapping, control, visualisation—is implemented from first principles, providing deep understanding of the SLAM pipeline.
2. **Physically principled noise model:** The velocity-dependent odometry noise (α_1 – α_4) is the standard model from probabilistic robotics, not an ad-hoc approximation.
3. **Correct sensor–map separation:** LiDAR reads from the true world but maps using the estimated pose, faithfully replicating the real-world information flow.
4. **Reproducibility:** Fixed random seed (42) ensures deterministic results.
5. **Dual visualisation:** Simultaneous ground-truth and estimated-map display makes odometry drift immediately visible and pedagogically valuable.

6.2 Limitations

1. **No pose correction (open-loop SLAM):** The system performs *mapping with odometry* but lacks the “simultaneous localisation” half of SLAM—there is no mechanism to correct the estimated pose using map observations. This means localisation error grows unboundedly.
2. **No log-odds clamping:** Cells can saturate to extreme values ($l \rightarrow \pm\infty$), which could cause numerical issues in extended runs and prevents adaptation to dynamic environments.

3. **Ray-casting resolution:** The 0.5 m step size for LiDAR ray-casting can miss thin obstacles (< 0.5 m). A smaller step or analytical ray–cell intersection would improve accuracy.
4. **Fixed exploration path:** The predefined waypoints do not adapt to the discovered map. An information-theoretic exploration strategy (e.g., frontier-based exploration) would improve coverage efficiency.
5. **No uncertainty representation for the pose:** The estimated pose is a single point estimate with no associated covariance, preventing the use of probabilistic fusion techniques (EKF, particle filters).
6. **Variable shadowing in `slam.py`:** The loop variable `i` in the `update` method shadows the outer loop variable, which is a minor code quality issue.

6.3 Comparison with Full SLAM Solutions

Feature	This Project	EKF-SLAM	FastSLAM	Graph-SLAM
Pose representation	Point estimate	Gaussian (μ, Σ)	Particle set	Factor graph node
Map representation	Occupancy grid	Landmark-based	Grid per particle	Occupancy grid
Pose correction	None	Kalman update	Particle reweighting	Graph optimisation
Loop closure	No	Limited	Yes (scan matching)	Yes (explicit)
Scalability	$O(n)$ per ray	$O(n^2)$ landmarks	$O(M \cdot K)$	$O(n^3)$ optimisation

7 Future Work & Extensions

7.1 Immediate Improvements

1. **Scan Matching (ICP / Correlative):** Aligning consecutive LiDAR scans to correct the estimated pose would dramatically reduce drift. The Iterative Closest Point (ICP) algorithm or Olson’s correlative scan matching could be integrated with minimal architectural changes.
2. **Log-Odds Clamping:** Adding bounds (e.g., $l \in [-5, 5]$) would prevent numerical saturation and allow the map to adapt if obstacles are moved.
3. **Extended Kalman Filter (EKF) Integration:** Augmenting the `Robot` class with a state covariance matrix and implementing the EKF prediction/update cycle would provide principled pose uncertainty tracking.

7.2 Advanced Extensions

4. **Particle Filter (FastSLAM):** Maintaining M pose hypotheses, each with its own occupancy grid, would provide a complete probabilistic SLAM solution. The existing `OccupancyGridMap` class could be instantiated per particle with minimal modification.

5. **Frontier-Based Exploration:** Replacing the fixed waypoint list with an autonomous exploration strategy that targets the boundaries between known-free and unknown space would enable full autonomous mapping.
6. **Loop Closure Detection:** When the robot revisits a previously mapped area, detecting the match (e.g., via scan correlation) and correcting the accumulated drift would complete the SLAM pipeline.
7. **3D Extension:** Extending the 2D grid to a 3D voxel representation (or using OctoMap) and replacing 2D LiDAR with a 3D point cloud would bring the system closer to real-world robotic applications.

8 Repository Structure

```
Msc/Robotics/
environment.py      # World model and LiDAR simulation
robot.py           # Differential-drive kinematics + odometry noise
slam.py            # Log-odds occupancy grid with Bresenham
main.py            # Simulation loop, controller, visualisation
visualization.html # Interactive browser-based demo
plots/             # Generated simulation snapshots
    step_0000.png ... step_1095.png
    final_map.png
pyproject.toml      # uv project config (numpy, matplotlib)
README.md
```

9 Conclusion

This project demonstrates a working occupancy grid mapping system driven by simulated differential-drive kinematics and LiDAR sensing. The implementation successfully constructs a recognisable map of the environment, clearly illustrating both the power of the log-odds Bayesian update and the limitations of pure dead-reckoning localisation. The progressive degradation of map quality due to uncorrected odometry drift provides a compelling, visual motivation for the pose-correction mechanisms (scan matching, particle filters, graph optimisation) that distinguish full SLAM from mere mapping.

The codebase is modular, well-documented, and ready for extension—the most impactful next step would be integrating scan matching to close the localisation loop and transform this from an open-loop mapping system into a true SLAM solution.

References

1. Thrun, S., Burgard, W. and Fox, D. (2005) *Probabilistic Robotics*. MIT Press.
2. Smith, R., Self, M. and Cheeseman, P. (1990) ‘Estimating Uncertain Spatial Relationships in Robotics’, in Cox, I.J. and Wilfong, G.T. (eds.) *Autonomous Robot Vehicles*. Springer, pp. 167–193.
3. Elfes, A. (1989) ‘Using occupancy grids for mobile robot perception and navigation’, *Computer*, 22(6), pp. 46–57.

4. Bresenham, J.E. (1965) ‘Algorithm for computer control of a digital plotter’, *IBM Systems Journal*, 4(1), pp. 25–30.
5. Grisetti, G., Stachniss, C. and Burgard, W. (2007) ‘Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters’, *IEEE Transactions on Robotics*, 23(1), pp. 34–46.
6. Olson, E.B. (2009) ‘Real-time correlative scan matching’, *2009 IEEE International Conference on Robotics and Automation*, pp. 4387–4393.